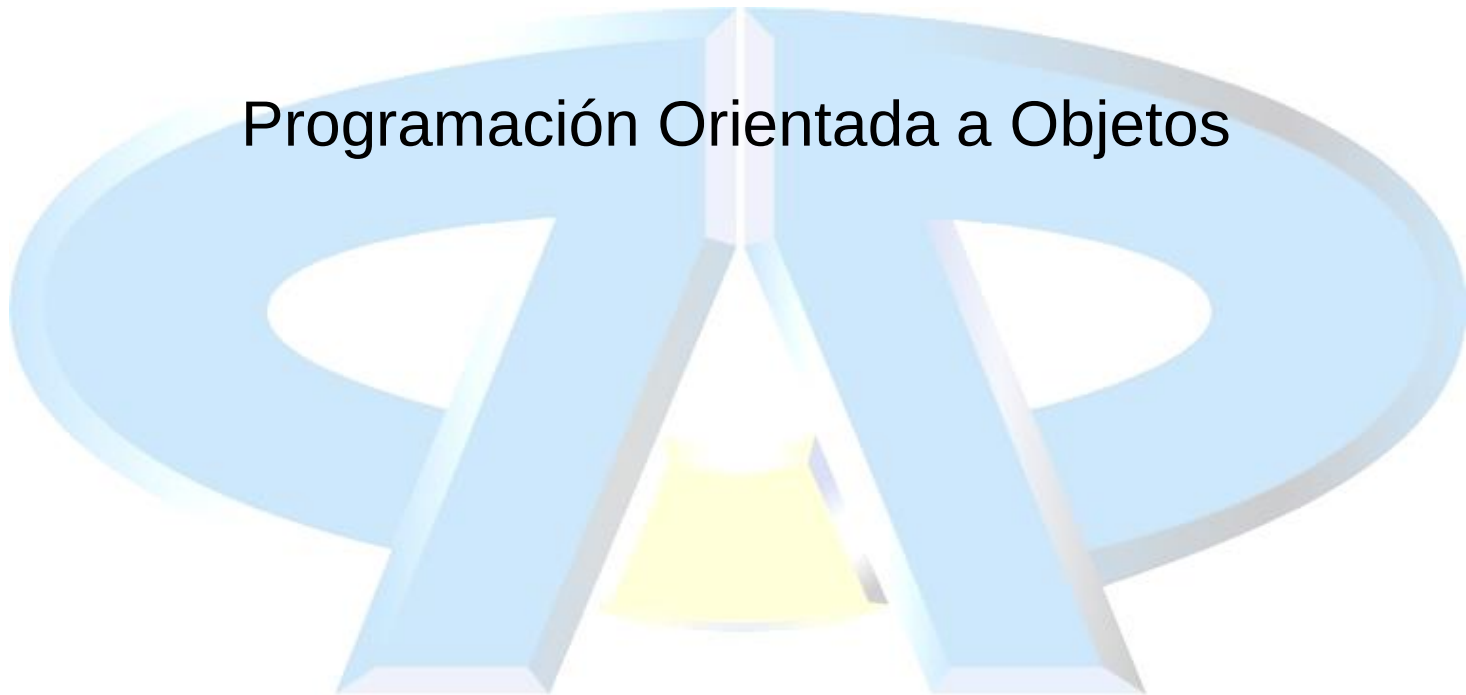


Programación Orientada a Objetos



Objetivos

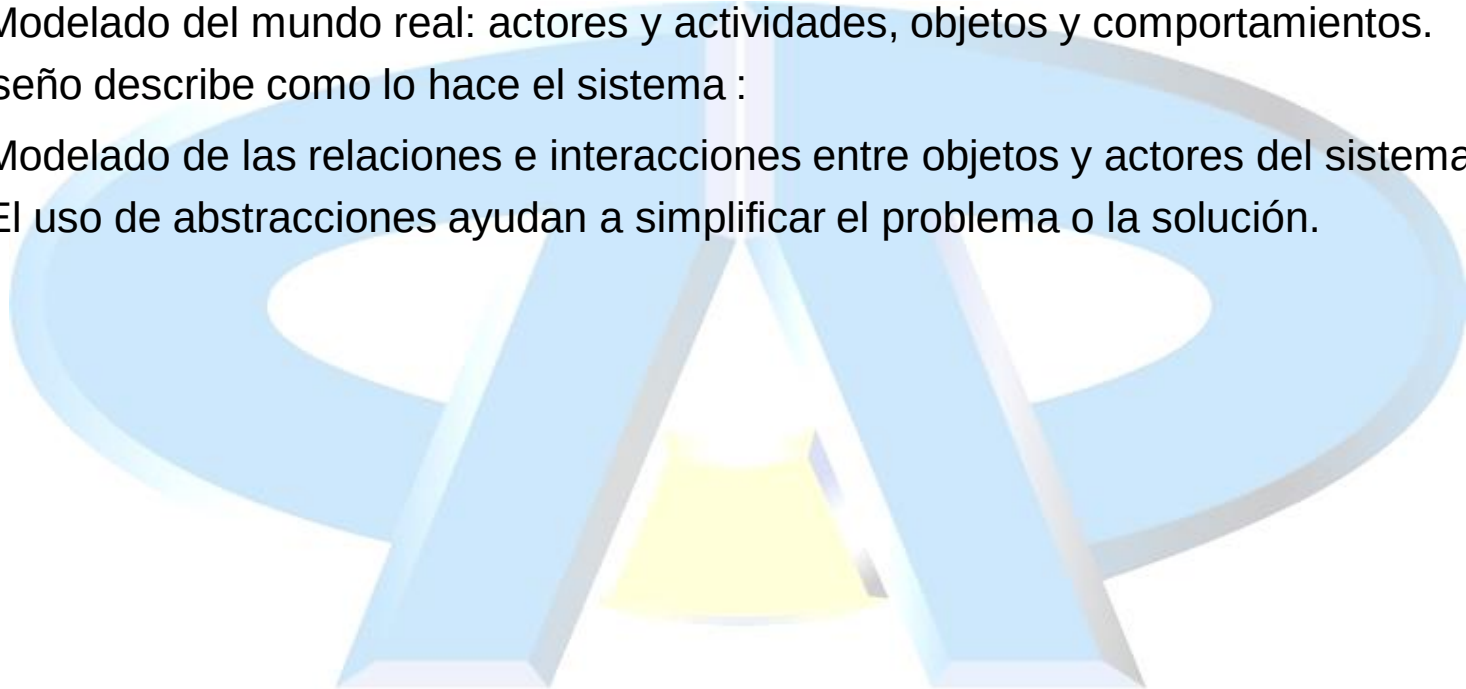
- Definir conceptos de modelado: abstracción, encapsulación y paquetes
- Determinar porque el código de las aplicaciones de tecnología Java es reutilizable
- Definir clase, miembro, atributo, método, constructor y paquete
- Utilizar los modificadores de acceso **private** y **public**
- Invocar a un método en un objeto particular
- Utilizar en línea de comando el API de tecnología a Java

Ingeniería del Software

- Evolución de las herramientas de desarrollo de software:
 - Lenguaje Máquina
 - Sistemas Operativos
 - Lenguajes de Programación de Alto Nivel
 - Librerías / APIs
 - Lenguajes de Programación Orientada a Objetos
 - Toolkits / Frameworks / APIs de Objetos
- Metodologías de desarrollo de software:
 - PUD: Proceso Unificado de Desarrollo de Software
 - Scrum: Desarrollo de Software Ágil

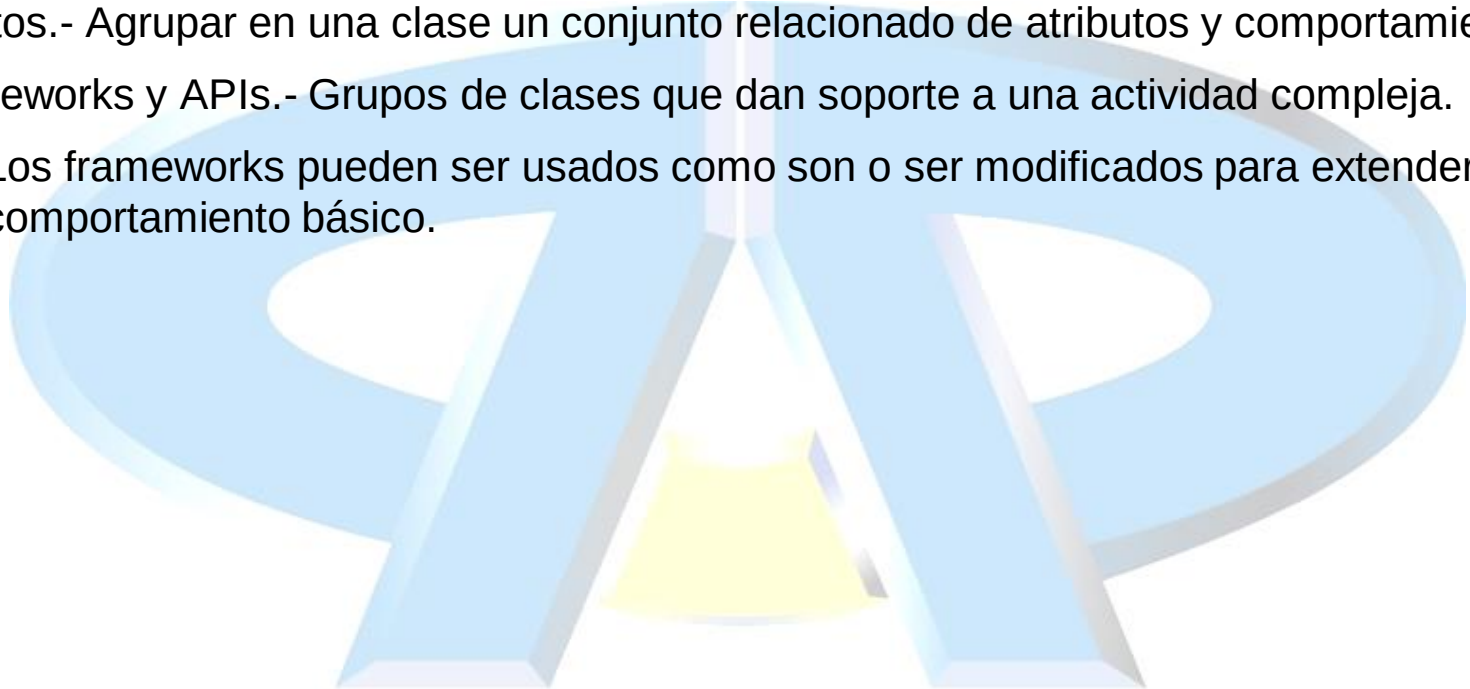
Análisis y Diseño.

- Análisis describe que necesita hacer el sistema.
 - Modelado del mundo real: actores y actividades, objetos y comportamientos.
- El Diseño describe como lo hace el sistema :
 - Modelado de las relaciones e interacciones entre objetos y actores del sistema.
 - El uso de abstracciones ayudan a simplificar el problema o la solución.



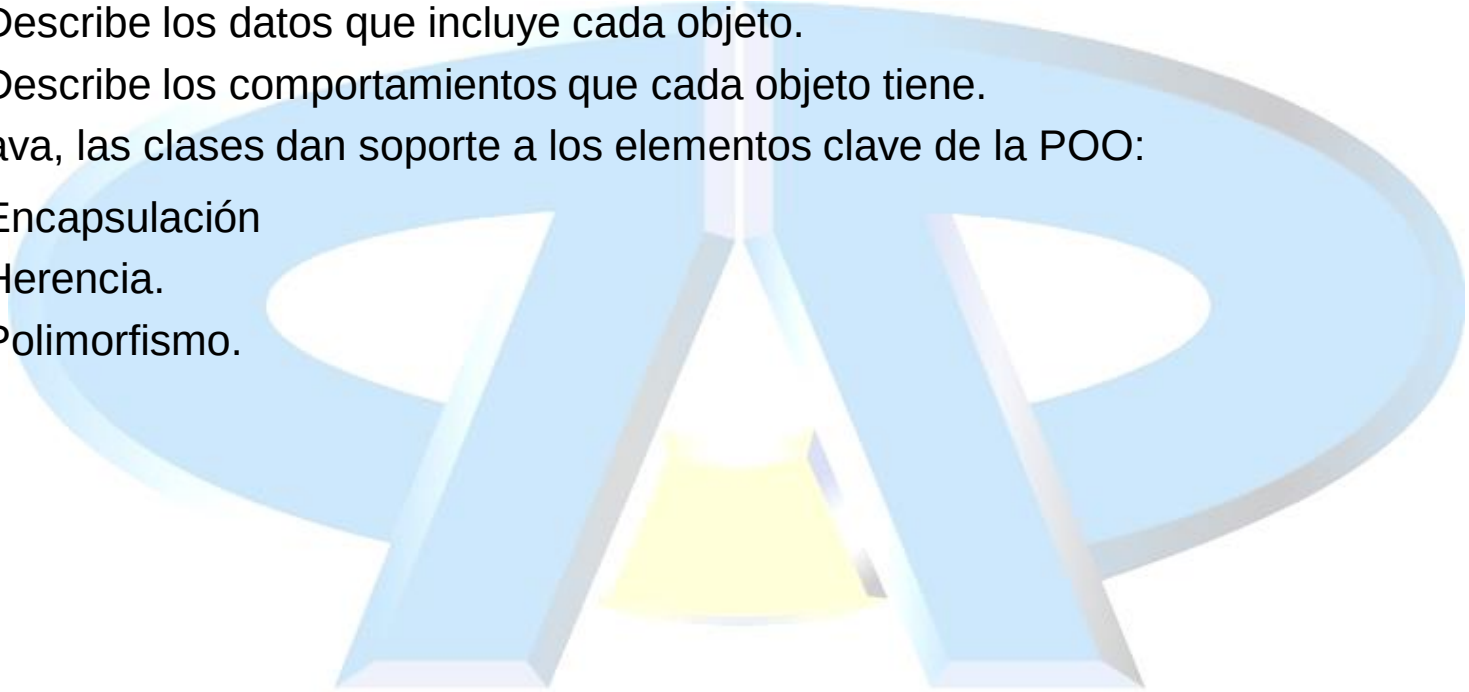
Abstracción

- Funciones.- Escribir un algoritmo una vez para ser usado en muchas situaciones.
- Objetos.- Agrupar en una clase un conjunto relacionado de atributos y comportamientos.
- Frameworks y APIs.- Grupos de clases que dan soporte a una actividad compleja.
 - Los frameworks pueden ser usados como son o ser modificados para extender su comportamiento básico.



Clases: plantilla de objetos

- Una clase es una descripción de un objeto.
 - Describe los datos que incluye cada objeto.
 - Describe los comportamientos que cada objeto tiene.
- En Java, las clases dan soporte a los elementos clave de la POO:
 - Encapsulación
 - Herencia.
 - Polimorfismo.



Declaración de clases.

- `<declaración_clase> ::=`

```
<modificador> class <nombre_clase> {  
    <declaración_atributos>*  
    <declaración_constructores>*  
    <declaración_métodos>*  
}
```

- Ejemplo:

```
public class Vehiculo {  
    private double carga;  
    public void setCarga(double kilos){  
        carga = kilos;  
    }  
}
```

Declaración de atributos

- Sintaxis básica de un atributo:

`<declaración_atributo> ::=`

`[<modificador>] <tipo> <identificador> [= valor_defecto];`

`<tipo> ::= byte | short | int | long | float | double | char | boolean | <nombre_clase>`

- Ejemplos:

```
public class Ejemplo {  
    private int a;  
    public float y = 1000.00;  
    private String s = "Java";  
}
```


Declaración de métodos

- Sintaxis básica de un método:

```
<declaración_método> ::=  
<modificador> <tipo_retorno> <identificador> (<args>*) {  
    <sentencias>*  
}
```

```
<args> ::= <tipo_argumento> <nombre_argumento> ,
```

- Ejemplo:

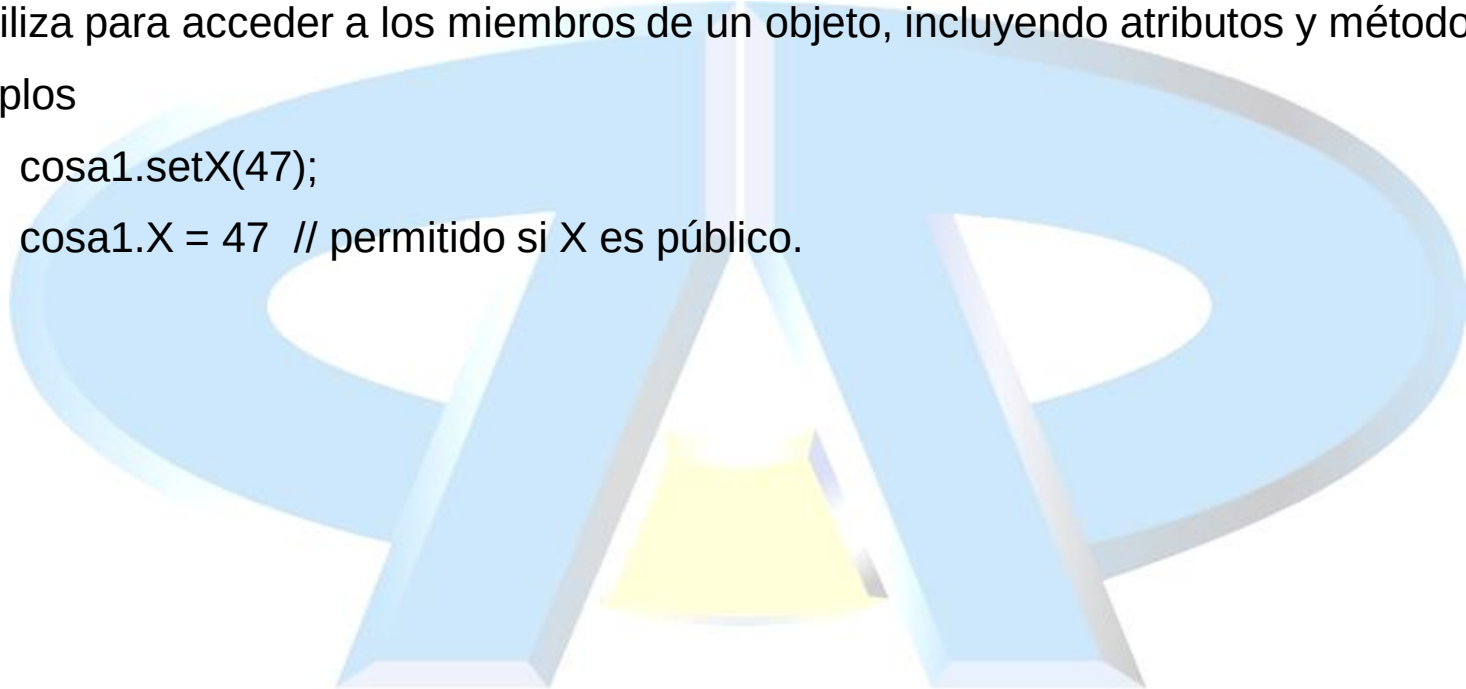
```
public class Cosa {  
    private int x;  
    public int getX(){  
        return x;  
    }  
    public void setX(int newX) {  
        x = newX;  
    }  
}
```

Acceso a objetos

- La notación “punto”: <objeto>.<miembro>
- Se utiliza para acceder a los miembros de un objeto, incluyendo atributos y métodos.
- Ejemplos

```
cosa1.setX(47);
```

```
cosa1.X = 47 // permitido si X es público.
```



Ocultar información

```
public class MiFecha {  
    public int dia;  
    public int mes;  
    public int año;  
}
```

```
MiFecha f = new MiFecha();
```

```
f.dia = 32; // día inválido;
```

```
f.dia = 30; f.mes = 2; // combinación inválida.
```

```
f.dia = f.dia + 1; //no existe control de desbordamiento
```



Ocultar información.

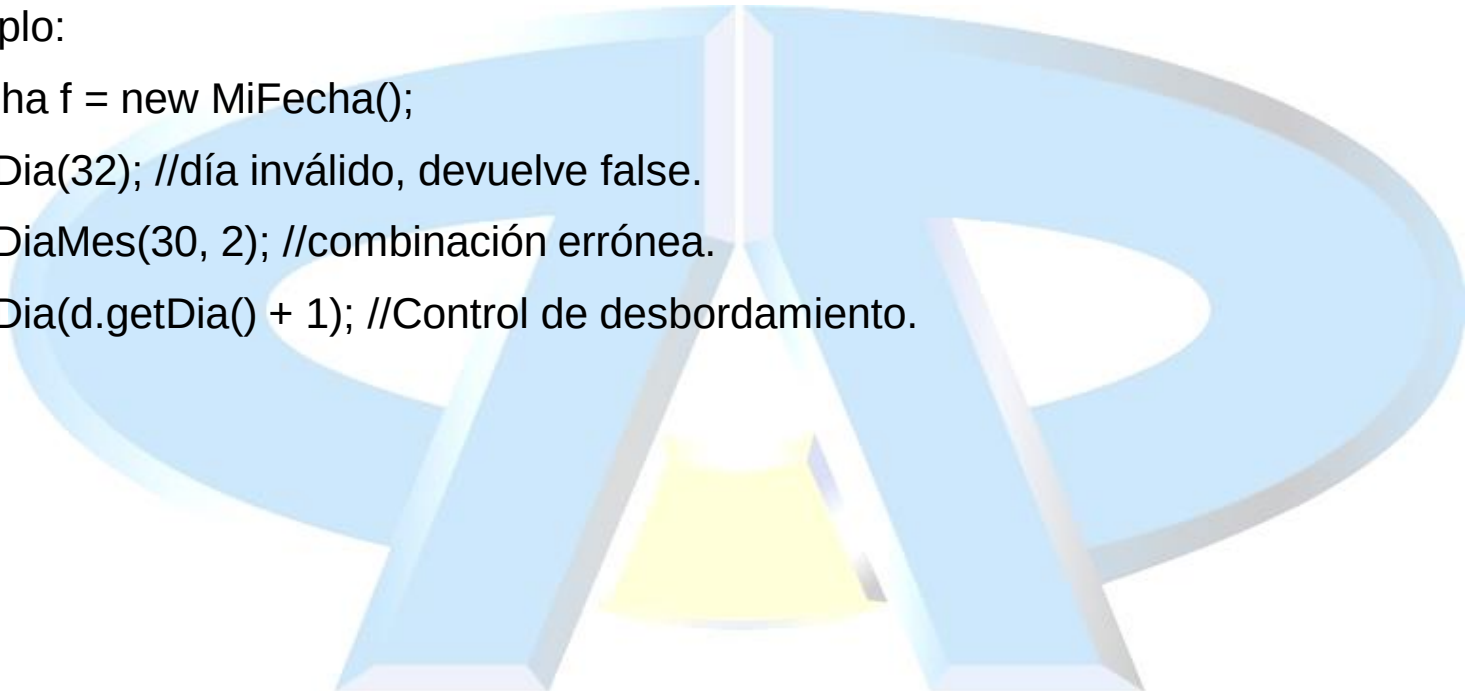
- El código cliente debe usar los métodos para acceder a los datos internos.
- Ejemplo:

```
Mifecha f = new MiFecha();
```

```
d.setDia(32); //día inválido, devuelve false.
```

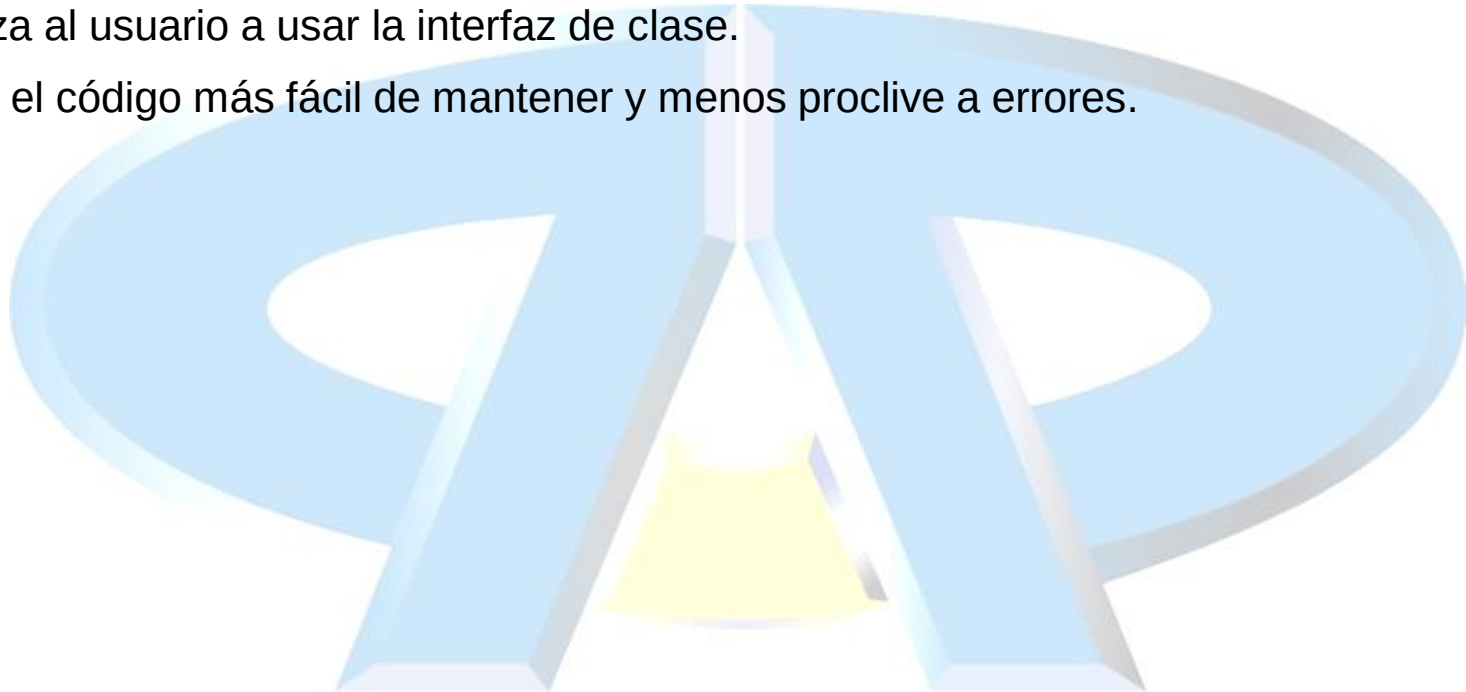
```
d.setDiaMes(30, 2); //combinación errónea.
```

```
d.setDia(d.getDia() + 1); //Control de desbordamiento.
```



Encapsulación

- Esconde los detalles de la implementación de una clase
- Fuerza al usuario a usar la interfaz de clase.
- Hace el código más fácil de mantener y menos proclive a errores.



Declaración de constructores

- Sintaxis básica de un constructor:

<declaración_constructor> ::=

```
[<modificador>] <nombre_clase> (<args>*) {  
    <sentencias>*  
}
```

- Ejemplo:

```
public class Cosa {  
    private int x;  
    public Cosa() {  
        x = 47;  
    }  
    public Cosa(int new_x){  
        x = new_x;  
    }  
}
```

El constructor por defecto

- Toda clase tiene por lo menos un constructor.
- Si no se especifica uno explícitamente, se añade automáticamente el constructor por defecto.
 - No tiene argumentos.
 - No tiene cuerpo.
- Posibilita crear instancias al ejecutar `new Xxx()` sin tener que escribir un constructor.
- Si se escribe cualquier tipo de “constructor”, el constructor por defecto no se añade.

Esquema del fichero fuente

- Sintaxis básica de un fichero fuente Java

`<fichero_fuente> ::=`

`[<declaración_paquete>]`

`<declaración_import>*`

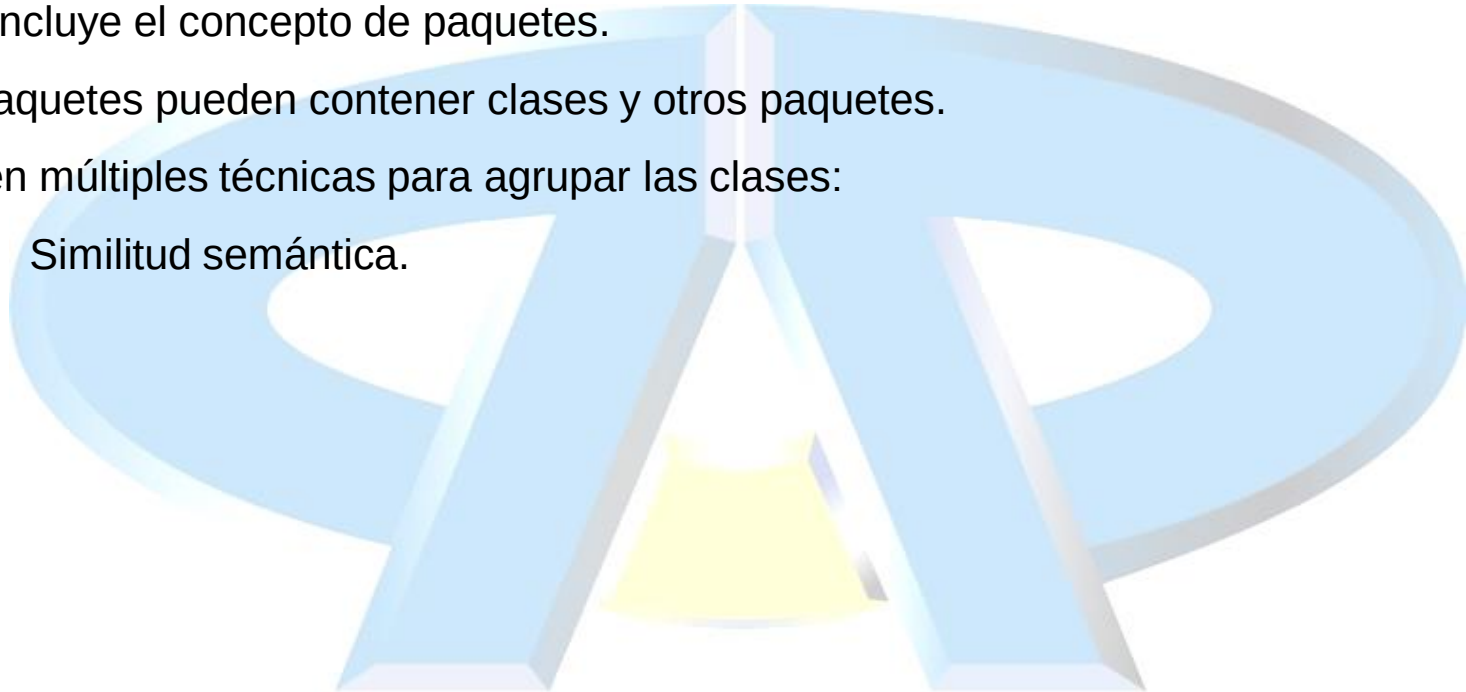
`<declaración_clase>+`

- Ejemplo:

```
package shipping.reports;
import shipping.domain.*;
import java.util.List;
import java.io.*;
public class VehicleCapacityReport {
    private List vehicles;
    public void generateReport(Writer output) {}
}
```


Paquetes de software.

- Agrupar las clases para facilitar la gestión del sistema.
- UML incluye el concepto de paquetes.
- Los paquetes pueden contener clases y otros paquetes.
- Existen múltiples técnicas para agrupar las clases:
 - Similitud semántica.



Sentencia *package*;

- Mecanismo de empaquetado.

`<package_declaration> ::= package <top_package_name>[.subpackage]*;`

- Para indicar que las clases contenidas en un fichero fuente pertenecen a un determinado paquete, se utiliza esta cláusula.

```
package shipping.domain;
```

```
Definición de la clase {
```

```
    ...
```

```
}
```

Sentencia *package*.

- Norma acerca de la cláusula *package*.
 - La declaración de paquete debe estar al comienzo del fichero fuente.
 - Solamente puede estar precedida por líneas en blanco o comentarios.
 - Sólo se permite una declaración de paquete por fichero fuente. Todas las clases del fichero fuente pertenecerán al mismo paquete.
 - Si un fichero fuente no tiene declaración de paquete, las clases pertenecen al paquete sin nombre o por defecto.

Sentencia *import*.

- Declaración:

`<import_declaration> ::= import <package_name>[.<subpackage>]*.<class_name | *>;`

- Sirve para indicar al compilador donde puede encontrar las clases.
- Las sentencias *import* van después de la sentencia *package*.
- No es necesario importar las clases que están en el mismo paquete, puesto que ya se conocen directamente.
- La cláusula *import* especifica la clase a la que se va a acceder. * indica todas las **clases** de un determinado *package*.
 - `import java.awt.*`
 - `import java.awt.event.*`
- La cláusula *import* especifica una ruta a la clase, pero no carga en memoria la propia clase, con lo que utilizar * no tiene aspectos demasiado nocivos a nivel de recursos.

Esquema de directorio y paquetes.

- Los paquetes son almacenados en un árbol de directorios que contiene ramas con los elementos del nombre del paquete.
- Organización de proyectos.
- Ubicación de archivos JAR.
 - `$JAVA_HOME/jre/lib/ext`
 - `%JAVA_HOME%\jre\lib\ext`

